

THE UNIVERSITY OF AZAD JAMMU & KASHMIR  
MUZAFFARABAD

---

DATABASE MANAGEMENT SYSTEM LAB

**Lab 12 — Aggregate Functions, GROUP BY and HAVING**  
**Retail & University Databases**

|                     |                                         |
|---------------------|-----------------------------------------|
| <b>Name</b>         | Tayyab Hanif Awan                       |
| <b>Roll No</b>      | 2024-SE-11                              |
| <b>Semester</b>     | 04                                      |
| <b>Course Title</b> | Database Management System (CS-2204)    |
| <b>Submitted To</b> | Engr. Awais Rathore                     |
| <b>Lab No.</b>      | 12                                      |
| <b>Tool Used</b>    | XAMPP (MySQL / MariaDB) with phpMyAdmin |

## 1. Objective

---

- Apply the 5 core aggregate functions: COUNT, SUM, AVG, MIN, and MAX.
- Group data using GROUP BY, and filter groups using HAVING.
- Understand the difference between HAVING (filters groups) and WHERE (filters rows).
- Combine aggregate queries with INNER JOIN and LEFT JOIN across multiple tables.

## 2. Database Structure

---

Two databases are used: `agg_lab` (a Retail/Order system) for guided practice, and `uni_lab` (a University system) for the graded assessment.

### 2.1 `agg_lab`

| Table     | Purpose                                         |
|-----------|-------------------------------------------------|
| Customer  | Customer details                                |
| Product   | Product catalog with category and price         |
| OrderItem | Order line items linking customers and products |

### 2.2 `uni_lab` (assessment)

| Table      | Purpose                                        |
|------------|------------------------------------------------|
| Student    | Student details, linked to a city              |
| Course     | Course details, linked to a department         |
| Enrollment | Links students to courses, with marks and fees |

## 3. Key Tasks / Concepts Implemented

---

- Task A1-A10: Whole-table aggregates — total counts, price extremes, average price, total stock, unique cities/categories, order date range, total revenue.
- Task B1-B7: GROUP BY reporting — customers by city, inventory by category, monthly order volume, high-value categories via HAVING.
- Task B9-B17: LEFT JOIN + aggregation to retain zero-activity customers/products, revenue per category, top clients, top-selling items.
- Assessment Q1-Q12 (`uni_lab`): student/course counts, performance averages per course, department revenue, top-performing students, and departments underperforming a mark threshold.

## 4. Implementation (phpMyAdmin)

All aggregate tasks and the University assessment were executed and verified using phpMyAdmin under XAMPP.

The screenshot displays the phpMyAdmin interface with two SQL queries executed. The top query counts records across three tables, and the bottom query finds the minimum and maximum prices in a product table.

**Query 1:**

```
-- Task A1: Counting total volume across core operational tables. SELECT (SELECT COUNT(*) FROM Customer) AS TotalCustomers, (SELECT COUNT(*) FROM Product) AS TotalProducts, (SELECT COUNT(*) FROM OrderItem) AS TotalOrders;
```

**Query results operations:** Print, Copy to clipboard, Export, Display chart, Create view

**Bookmark this SQL query**

**Showing rows 0 - 0 (1 total, Query took 0.0006 seconds.)**

```
SELECT MIN(Price) AS MinPrice, MAX(Price) AS MaxPrice FROM Product;
```

☐ Profiling [ Edit inline ] [ Edit ] [ Explain SQL ] [ Create PHP code ] [ Refresh ]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

**Extra options**

| MinPrice | MaxPrice  |
|----------|-----------|
| 350.00   | 185000.00 |

☐ Show all | Number of rows: 25 | Filter rows: Search this table

✓ Showing rows 0 - 0 (1 total, Query took 0.0003 seconds.)

```
SELECT ROUND(AVG(Price), 2) AS AvgPrice FROM Product;
```

☐ Profiling [ [Edit inline](#) ] [ [Edit](#) ] [ [Explain SQL](#) ] [ [Create PHP code](#) ] [ [Refresh](#) ]

☐ Show all | Number of rows: 25 ▾ Filter rows:

Extra options

**AvgPrice**

26265.05

☐ Show all | Number of rows: 25 ▾ Filter rows:

✓ Showing rows 0 - 0 (1 total, Query took 0.0003 seconds.)

```
SELECT SUM(StockQty) AS TotalStock FROM Product;
```

☐ Profiling [ [Edit inline](#) ] [ [Edit](#) ] [ [Explain SQL](#) ] [ [Create PHP code](#) ] [ [Refresh](#) ]

☐ Show all | Number of rows: 25 ▾ Filter rows:

Extra options

**TotalStock**

618

☐ Show all | Number of rows: 25 ▾ Filter rows:

Queries of WHOLE-TABLE AGGREGATES & COUNT VARIATIONS

✓ Showing rows 0 - 1 (2 total, Query took 0.0011 seconds.)

```
SELECT Category, ROUND(AVG(Price), 2) AS AvgPrice FROM Product GROUP BY Category HAVING AVG(Price) > 5000;
```

☐ Profiling [ [Edit inline](#) ] [ [Edit](#) ] [ [Explain SQL](#) ] [ [Create PHP code](#) ] [ [Refresh](#) ]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

← T → Category AvgPrice

|                          |      |      |        |             |          |
|--------------------------|------|------|--------|-------------|----------|
| <input type="checkbox"/> | Edit | Copy | Delete | Electronics | 48950.00 |
| <input type="checkbox"/> | Edit | Copy | Delete | Furniture   | 31750.25 |

↑ ☐ Check all | With selected: Edit Copy Delete Export

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Query results operations

GROUP BY and Having query

Your SQL query has been executed successfully.

```
SELECT (SELECT COUNT(*) FROM Student) AS TotalStudents, (SELECT COUNT(*) FROM Course) AS TotalCourses;
```

☐ Profiling [ [Edit inline](#) ] [ [Edit](#) ] [ [Explain SQL](#) ] [ [Create PHP code](#) ] [ [Refresh](#) ]

Extra options

| TotalStudents | TotalCourses |
|---------------|--------------|
| 9             | 6            |

Query results operations

Your SQL query has been executed successfully.

```
SELECT COUNT(DISTINCT City) AS DistinctCities FROM Student WHERE City IS NOT NULL;
```

☐ Profiling [ [Edit inline](#) ] [ [Edit](#) ] [ [Explain SQL](#) ] [ [Create PHP code](#) ] [ [Refresh](#) ]

Extra options

**DistinctCities**

3

Query results operations



Print



Copy to clipboard



Export



Display chart



Create view

✓ Showing rows 0 - 0 (1 total, Query took 0.0003 seconds.)

```
SELECT ROUND(AVG(Marks), 2) AS AvgMarks, MIN(Marks) AS MinMarks, MAX(Marks) AS MaxMarks FROM Enrollment;
```

☐ Profiling [ [Edit inline](#) ] [ [Edit](#) ] [ [Explain SQL](#) ] [ [Create PHP code](#) ] [ [Refresh](#) ]



Show all

Number of rows:

25



Filter rows:

Search this table

Extra options

| AvgMarks | MinMarks | MaxMarks |
|----------|----------|----------|
|----------|----------|----------|

77.56

55

95



Show all

Number of rows:

25



Filter rows:

Search this table

Query results operations

```
SELECT City, COUNT(StudentID) AS NumStudents FROM Student GROUP BY City ORDER BY City IS NULL ASC, NumStudents DESC;
```

☐ Profiling [ [Edit inline](#) ] [ [Edit](#) ] [ [Explain SQL](#) ] [ [Create PHP code](#) ] [ [Refresh](#) ]

☐ Show all | Number of rows: 25 ▾ Filter rows:

Extra options

| City      | NumStudents ▾ 2 |
|-----------|-----------------|
| Lahore    | 4               |
| Islamabad | 2               |
| Karachi   | 2               |
| NULL      | 1               |

☐ Show all | Number of rows: 25 ▾ Filter rows:

```
SELECT Department, COUNT(CourseID) AS NumCourses FROM Course GROUP BY Department ORDER BY NumCourses DESC;
```

☐ Profiling [ [Edit inline](#) ] [ [Edit](#) ] [ [Explain SQL](#) ] [ [Create PHP code](#) ] [ [Refresh](#) ]

☐ Show all | Number of rows: 25 ▾ Filter rows:

Extra options

| Department       | NumCourses ▾ 1 |
|------------------|----------------|
| Computer Science | 3              |
| Business         | 1              |
| Electrical Engg  | 1              |
| Mathematics      | 1              |

☐ Show all | Number of rows: 25 ▾ Filter rows:

5 screenshots attached, of queries from the assessment problem in lab # 12 manual

## 5. Conclusion

This lab tied together everything from earlier labs — filtering, joins, and now aggregation — to produce genuine summary-level business and academic reports, the most common real-world use of SQL.